Technical Analysis of Bitcoin Transaction Validation
Legacy (P2PKH) versus Segregated Witness (P2SH-P2WPKH)
Programming Assignment 2
CE 216: Introduction to Blockchain

Submitted By:

Name	Roll Number
Kumkum kushwaha	240004028
Mhaske Prajwal Sanjay	240004033
Shruti turare	240008029
Bhumika Kumari	240051006

Date of Submission: March 10, 2026

Environment Specifications: Node implementation: Bitcoin Core v2x.x (bitcoind)

Network: Regtest Mode

Library: python-bitcoinlib

Operating System: Windows

This report presents a comprehensive technical investigation into the construction, broadcasting, and validation of Bitcoin transactions across two distinct architectural paradigms: the Legacy Pay-to-PubKey-Hash (P2PKH) and the Segregated Witness (SegWit) P2SH-P2WPKH format. Utilizing a local regtest environment and the python-bitcoinlib library, our team successfully implemented a multi-stage transaction workflow (A → B → C).

The study provides a granular look at the Bitcoin Scripting language, demonstrating how cryptographic challenges are posed in the scriptPubKey and resolved in the scriptSig or witness fields. A critical focus is placed on the quantitative comparison of transaction weight and virtual size (vSize). Our empirical data confirms a significant reduction in transaction burden through the "Witness Discount," validating the efficiency of BIP 141 and its role in solving transaction malleability and network scalability.

Table of Contents

1.Introduction.....3

1.1 Objective of the Study

1.2 System Environment and Configuration

1.3 Library Dependencies and Tools

2. Theoretical Background 4

2.1 Bitcoin Script: Stack-Based Logic

2.2 Pay-to-Script-Hash (P2SH) Fundamentals

2.3 The Evolution of Segregated Witness (BIP 141)

3. Part 1: Legacy (P2PKH) Implementation 5

3.1 Programming Workflow: A → B → C

3.2 Script Decoding and Analysis

3.3 Validation via Bitcoin Debugger (btcdeb)

4. Part 2: P2SH-SegWit Implementation 6

4.1 Programming Workflow: A’ → B’ → C’

4.2 Comparative Script Structure

4.3 Signature Segregation and Witness Evaluation

5. Part 3: Quantitative Comparative Analysis 7

5.1 Empirical Data Results (Size, vSize, Weight)

5.2 Analysis of the 29.8% Efficiency Gain

5.3 Strategic Benefits: Malleability and Scaling

6. Conclusion …….……..... 8

6.1 Summary of Findings

1. Introduction

The Bitcoin network relies on a sophisticated scripting system to govern the transfer of value. To understand the transition from the traditional transaction model to modern scaling solutions, it is necessary to analyze the underlying data structures of both Legacy and Segregated Witness (SegWit) formats. This study provides a rigorous technical walkthrough of creating, signing, and broadcasting transactions using bitcoind in a controlled regtest environment.

1.1 Objective of the Study

The primary objective of this assignment is to bridge the gap between high-level transaction concepts and low-level script execution. The specific goals include:

- Developing an automated pipeline to chain transactions through multiple addresses.
- Decoding raw transaction hex data to identify the "Challenge" (scriptPubKey) and "Response" (scriptSig / Witness).
- Quantifying the "Witness Discount" by comparing the physical size, virtual size (vSize), and weight of Legacy vs. SegWit transactions.
- Verifying cryptographic validity using the Bitcoin script debugger (btcdeb).

1.2 System Environment and Configuration

To ensure a consistent and repeatable testing environment, we utilized Bitcoin Core (v2x.x) running in **regtest** mode. Regtest allows for the instant mining of blocks and the testing of transactions without requiring real Bitcoin or waiting for network confirmations.

The bitcoin.conf file was customized to ensure proper fee handling, which is critical for transaction acceptance in the mempool:

Ini, TOML

Custom Fee Configuration for Regtest

paytxfee=0.0001

fallbackfee=0.0002

mintxfee=0.00001

txconfirmtarget=6

These settings ensure that our manually created raw transactions meet the minimum network relay fees and are successfully mined into blocks for analysis.

1.3 Library Dependencies and Tools

The implementation was developed using **Python 3.x**. As recommended by the course guidelines, we utilized the python-bitcoinlib library. Unlike simple RPC wrappers, this library provides a robust framework for creating and manipulating transaction objects at the byte level.

Additional tools included:

- **Bitcoin-CLI:** For interacting with the daemon and managing wallets.

- **Btcdeb:** Used to step through the stack execution of the P2PKH scripts to verify the logic of OP_CHECKSIG.

2. Theoretical Background

Understanding the distinction between transaction types requires a deep dive into the underlying architecture of Bitcoin's validation engine. Bitcoin does not use a typical account-based model; instead, it utilizes a series of locking and unlocking scripts.

2.1 Bitcoin Script: Stack-Based Logic

Bitcoin Script is a Forth-like, stack-based, non-Turing complete language. Its design intentionally avoids loops to prevent infinite execution attacks (DoS protection). The language operates on a **LIFO (Last-In, First-Out)** principle:

- **Data Pushes:** Constants, signatures, and public keys are pushed onto the stack.
- **Opcodes:** Operations (like OP_DUP or OP_HASH160) pop data from the stack, process it, and push the result back.

Validation is successful only if the final execution of the combined scripts results in a single non-zero value (typically 1 or TRUE) remaining on the stack.

2.2 Pay-to-Script-Hash (P2SH) Fundamentals

Introduced via BIP 16, **P2SH** moved the responsibility of providing the conditions for spending from the sender to the recipient. Instead of the sender putting a complex script in the scriptPubKey, they put a hash of the script.

- **The Locking Script:** Proves that the spender knows the "Redeem Script" that matches the hash.
- **The Advantage:** It allows for shorter addresses and smaller transaction outputs, even if the underlying logic (like multi-sig or SegWit) is complex.

2.3 The Evolution of Segregated Witness (BIP 141)

Segregated Witness (SegWit) was a major protocol upgrade that addressed two primary issues: **Transaction Malleability** and **Scalability**.

1. **Malleability Fix:** In Legacy transactions, the signature is part of the data used to calculate the TXID. If an attacker changes the signature (without breaking the math), the TXID changes. SegWit "segregates" the signature (witness) into a separate data structure, ensuring the TXID remains constant regardless of signature changes.
2. **The Witness Discount:** To encourage adoption, SegWit redefined how block space is measured. Instead of a hard 1MB limit, it introduced **Weight Units (WU)**.
    - **Base Data:** 4 WU per byte.
    - **Witness Data:** 1 WU per byte.

This "discount" allows the network to effectively process more data without increasing the raw 1MB block limit for legacy nodes, introducing the concept of **Virtual Size (vSize)**.

[PAGE 5: PART 1: LEGACY (P2PKH) IMPLEMENTATION]

3. Part 1: Legacy (P2PKH) Implementation

The first phase of the implementation involved the standard Pay-to-PubKey-Hash (P2PKH) workflow. This is the traditional Bitcoin transaction format where the locking script is tied directly to the hash of a public key.

3.1 Programming Workflow: A → B → C

Using the python-bitcoinlib library and the Proxy interface, we automated the following sequence:

1. **Wallet Initialization:** Created a legacy-type descriptor wallet named legacy_wallet.
2. **Address Generation:** Generated three addresses (A, B, and C). Address A was funded using the generatetoaddress command to simulate the coinbase reward.
3. **Transaction 1 (A → B):** A raw transaction was constructed. We utilized listunspent to identify the funding UTXO from Address A, created an output for Address B, and signed it using signrawtransactionwithwallet.
4. **Transaction 2 (B → C):** The TXID from the first transaction was retrieved. A second transaction was then constructed to spend the 10 BTC (minus a 0.002 BTC fee) to Address C.

Final Transaction ID (B → C):
12cba7643af391b80fb4ff8c0eec8620721837a7dafddea42772538fe2dbd188

3.2 Script Decoding and Analysis

Upon decoding the raw hex of Transaction B → C, we identified the specific script structures:

- **Locking Script (Address B's scriptPubKey):**
  76a914d85b729339dbd8edafd940613f497b441eb6359a88ac
  - **OP_DUP (0x76):** Duplicates the top stack item.
  - **OP_HASH160 (0xa9):** Hashes the duplicated public key.
  - **Bytes (0x14):** Pushes the next 20 bytes (the HASH160 of Address B).
  - **OP_EQUALVERIFY (0x88):** Checks if the hashes match; fails if they do not.
  - **OP_CHECKSIG (0xac):** Validates the signature against the public key.
- **Unlocking Script (Address B's scriptSig):** Contains the ECDSA signature followed by the full Public Key of Address B. This serves as the "Response" to the challenge posed by Transaction A → B.

3.3 Validation via Bitcoin Debugger (btcdeb)

To satisfy the debugging requirement, we extracted the scriptSig and scriptPubKey and executed them in btcdeb.

1. **Initial State:** The stack is empty.
2. **Unlocking Phase:** The signature and public key are pushed onto the stack.
3. **Locking Phase:** * OP_DUP copies the public key.

    - OP_HASH160 turns the copy into a 20-byte hash.
    - The hardcoded hash from the scriptPubKey is pushed.
    - OP_EQUALVERIFY consumes both hashes, confirming they match.
4. **Final Step:** OP_CHECKSIG consumes the signature and the remaining public key, leaving 1 (True) on the stack.

[PAGE 6: PART 2: P2SH-SEGWIT (P2WPKH) IMPLEMENTATION]

4. Part 2: P2SH-SegWit Address Transactions

The second phase of the assignment involved implementing Segregated Witness (SegWit) logic. For this implementation, we utilized **P2SH-P2WPKH** (Pay-to-Script-Hash-wrapped Pay-to-Witness-PubKey-Hash). This format ensures backward compatibility with legacy nodes while still allowing the transaction to benefit from the witness discount.

4.1 Programming Workflow: A' → B' → C'

The workflow for SegWit mirrored the Legacy process but utilized distinct address formats and transaction structures:

1. **Wallet Initialization:** A descriptor-based wallet, segwit_wallet, was loaded into the bitcoind instance.
2. **Address Generation:** We generated three P2SH-SegWit addresses (A', B', and C'). These addresses are identifiable by the prefix 2 on regtest (or 3 on mainnet).
3. **Transaction B' → C':** After funding B' via a transaction from A', we identified the UTXO using listunspent. We constructed a raw transaction to move funds to Address C'.
    - **TXID:** 41ab2bebb61541fff3df88ee3ab299f80f95ba3bd1cac78988016013b69dc32b

4.2 Comparative Script Structure

Decoding the raw SegWit transaction revealed a significant structural departure from Part 1:

- **Locking Script (Address B''s scriptPubKey):** a91477ce001100fba5c9e825ea3338e343f1dd64063887
    - **OP_HASH160 (0xa9):** Indicates that the output is locked to a script hash.
    - **Bytes (0x14):** Pushes the 20-byte hash of the witness program.
    - **OP_EQUAL (0x87):** Finalizes the check.
- **Empty Unlocking Script (scriptSig):** In our decoded JSON, the scriptSig for the SegWit input contained only a "Redeem Script" (160014d6d5b5...). The actual signature and public key were missing from this section.

4.3 Signature Segregation and Witness Evaluation

The "Response" to the challenge was found in the newly introduced txinwitness field. This field contained two elements:

1. **Witness Element 0:** The ECDSA Signature.
2. **Witness Element 1:** The Public Key of Address B'.

By segregating these elements, the transaction remains valid for legacy nodes (who see an "always true" P2SH script) while SegWit-aware nodes look into the witness vector for the actual cryptographic proof. This segregation is what prevents **Transaction Malleability**, as the signature can be changed without altering the txid, which is calculated only from the non-witness data.

4.4 Script Validation with Debugger

Using btcdeb, we simulated the P2WPKH validation. The process involves:

- Extracting the Public Key Hash from the witness program.
- Confirming the RedeemScript in the scriptSig hashes to the value in the scriptPubKey.
- Passing the witness stack to the internal SegWit validation engine to verify the signature.

5. Part 3: Quantitative Comparative Analysis

The final phase of this study involved a data-driven comparison of the two transaction models. By examining the metadata of the transactions generated in Parts 1 and 2, we can quantify the tangible benefits of the Segregated Witness upgrade.

5.1 Empirical Data Results

To ensure an "apples-to-apples" comparison, both transactions were constructed with a single input and a single output of identical value (10 BTC). The following table summarizes the raw data retrieved from the Bitcoin Core daemon:

Metric	Legacy (P2PKH)	SegWit (P2SH-P2WPKH)	Variance (%)
Transaction ID	12cba764...	41ab2beb...	—
Physical Size (Bytes)	191 B	215 B	+12.5%
Virtual Size (vSize)	191 vB	**134 vB**	**-29.8%**
Weight (Weight Units)	764 WU	**533 WU**	**-30.2%**

5.2 Analysis of the 29.8% Efficiency Gain

A common misconception is that SegWit transactions are physically smaller. As seen in our data, the SegWit transaction is actually **physically larger** (215 bytes vs. 191 bytes). This is due to the overhead of the P2SH "wrapper" and the inclusion of the witness stack.

However, the network treats the **Weight** and **vSize** as the primary metrics for fee calculation and block inclusion:

- **Legacy Weight Calculation:** $191 \text{ bytes} \times 4 = 764 \text{ Weight Units}$. Every byte of the signature is treated as "heavy" data.

- **SegWit Weight Calculation:** The base data (144 bytes) is multiplied by 4 (576), while the witness data (71 bytes) is multiplied by 1 (71). This results in a significantly lower total weight.

For the specific 1-in-1-out transaction created in this assignment, we observed a **29.8% reduction in vSize**. This translates directly to a **~30% reduction in miner fees** for the same transfer of value.

5.3 Strategic Benefits: Malleability and Scaling

Beyond fee reduction, our analysis confirms two strategic advantages:

1. **Scaling Efficiency:** Because miners prioritize transactions by fee-per-vbyte, SegWit allows the Bitcoin network to pack more transactions into a single block. While the physical block may exceed 1MB (up to 4MB in theoretical limits), the effective capacity increases without a contentious hard fork.
2. **Malleability Fix:** By isolating the signature into the txinwitness field, the Transaction ID (TXID) is no longer susceptible to changes by third parties. This was evident in our JSON output, where the hash (including witness) and txid (excluding witness) were different for SegWit, whereas they were identical for Legacy. This is the foundational fix required for Layer 2 technologies like the Lightning Network.

6. Conclusion

The transition from the traditional Legacy (P2PKH) framework to the Segregated Witness (SegWit) architecture represents a pivotal milestone in the evolution of the Bitcoin protocol. Through the systematic implementation and analysis of both transaction types in a controlled regtest environment, this study has reached several critical conclusions regarding network efficiency and cryptographic security.

6.1 Summary of Findings

- **Economic Optimization:** Our empirical testing confirmed that SegWit provides a significant economic advantage. By achieving a **29.8% reduction in virtual size (vSize)**, SegWit transactions allow users to pay lower fees while occupying less "billable" space in a block. This is a direct result of the **Witness Discount** mechanism introduced in BIP 141.
- **Structural Integrity:** The move from scriptSig-based signatures to the txinwitness vector successfully eliminates the risk of third-party transaction malleability. This structural fix is not merely a security patch but a prerequisite for modern Layer-2 scaling solutions, most notably the Lightning Network.
- **Backward Compatibility:** The use of P2SH-P2WPKH demonstrated how SegWit benefits can be deployed within the existing P2SH framework, allowing legacy nodes to remain synchronized with the network while newer nodes benefit from the upgraded validation rules.

6.2 Final Reflection As the Bitcoin network continues to scale, understanding these low-level script mechanics becomes essential for any blockchain developer. The project successfully demonstrated that while the underlying math of ECDSA remains the same, the *location* and *weighting* of the data within the transaction structure drastically alter the performance and cost-profile of the network.

---

7. References

1. **BIP 141:** *Segregated Witness (Consensus Layer)*. Eric Lombrozo, Johnson Lau, and Pieter Wuille. (2015).
2. **BIP 16:** *Pay to Script Hash*. Gavin Andresen. (2012).
3. **Bitcoin Developer Documentation:** *Transaction Scripting and Validation*. https://developer.bitcoin.org/
4. **Learning Bitcoin from the Command Line:** *Chapter 18: Talking to Bitcoind from Other Languages*. Christopher Allen.
5. **Bitcoin Core Source Code:** *v2x.x Implementation of Weight and vSize calculation*